

You already have my resume. Use my resume only to understand my experience, projects, technologies, and background.

For the interview preparation, focus ONLY on technical concepts and explain them from

****Basic to Advanced level****.

Do not repeat my resume details unless a concept needs a project-based example.

1. Core Java – Basic to Advanced

Java Basics

- * What is Java
- * Features of Java
- * JDK
- * JRE
- * JVM
- * Bytecode
- * Compilation and execution process
- * Platform independence
- * Java architecture
- * Class
- * Object
- * Method
- * Variable
- * Keywords
- * Access modifiers
- * Packages
- * Static
- * Final
- * This
- * Super

OOP

- * Class and Object
- * Encapsulation
- * Inheritance
- * Polymorphism
- * Abstraction

- * Method Overloading
- * Method Overriding
- * Association
- * Aggregation
- * Composition
- * IS-A
- * HAS-A

Interface and Abstract Class

- * Interface
- * Abstract class
- * Interface vs Abstract class
- * Multiple inheritance using interface
- * Default methods
- * Static methods in interface
- * Functional interface
- * Marker interface

Constructors

- * Default constructor
- * Parameterized constructor
- * Constructor overloading
- * Constructor chaining
- * this()
- * super()

String

- * String
- * String pool
- * String immutability
- * String vs StringBuilder
- * StringBuilder vs StringBuffer
- * equals()
- * ==
- * hashCode()

Exception Handling

- * Exception
- * Error
- * Checked exception
- * Unchecked exception
- * RuntimeException
- * try-catch
- * finally
- * throw
- * throws
- * Custom exception
- * Exception propagation
- * Multiple catch
- * Global exception concept

Collections

- * Collection framework
- * List
- * Set
- * Map
- * ArrayList
- * LinkedList
- * Vector
- * Stack
- * HashSet
- * LinkedHashSet
- * TreeSet
- * HashMap
- * LinkedHashMap
- * TreeMap
- * Hashtable
- * ConcurrentHashMap
- * Comparable
- * Comparator
- * Iterator

- * ListIterator

HashMap

- * Internal working of HashMap

- * Hashing

- * hashCode()

- * equals()

- * Collision

- * Bucket

- * Load factor

- * Initial capacity

- * Treeification

- * HashMap vs ConcurrentHashMap

Java Generics

- * Generic class

- * Generic method

- * Type safety

- * Wildcards

- * ? extends

- * ? super

Java 8+ Features

- * Lambda expression

- * Functional interface

- * Predicate

- * Function

- * Consumer

- * Supplier

- * BiFunction

- * Method reference

- * Stream API

- * Optional

- * Default methods

- * Static interface methods

Stream API

- * filter()
- * map()
- * flatMap()
- * sorted()
- * distinct()
- * limit()
- * skip()
- * reduce()
- * collect()
- * groupingBy()
- * partitioningBy()
- * counting()
- * joining()
- * max()
- * min()
- * findFirst()
- * findAny()
- * anyMatch()
- * allMatch()
- * noneMatch()

Optional

- * Optional
- * of()
- * ofNullable()
- * empty()
- * isPresent()
- * ifPresent()
- * map()
- * filter()
- * orElse()
- * orElseGet()
- * orElseThrow()

Multithreading

- * Process vs Thread
- * Thread
- * Runnable
- * Callable
- * Future
- * ExecutorService
- * Thread Pool
- * Synchronization
- * synchronized
- * volatile
- * Atomic classes
- * Race condition
- * Deadlock
- * Starvation
- * Thread safety
- * Concurrent collections
- * CompletableFuture

JVM

- * JVM architecture
- * Class Loader
- * Heap
- * Stack
- * Method Area
- * Metaspace
- * Program Counter
- * Garbage Collection
- * Minor GC
- * Major GC
- * Full GC
- * JIT compiler
- * Memory leaks
- * OutOfMemoryError
- * StackOverflowError

2. Programming / Coding – Basic to Advanced

Prepare coding questions in Java 17/21.

Basic Programs

- * Even/Odd
- * Prime number
- * Factorial
- * Fibonacci
- * Palindrome
- * Armstrong number
- * Reverse number
- * Sum of digits
- * GCD
- * LCM
- * Swap numbers
- * Largest of three numbers
- * Leap year
- * Vowel/consonant

String Programs

- * Reverse String
- * Palindrome String
- * Count characters
- * Count vowels
- * Count words
- * Duplicate characters
- * Remove duplicate characters
- * First non-repeated character
- * First repeated character
- * Anagram
- * Reverse each word
- * Reverse complete sentence
- * Character frequency
- * String compression

- * Longest word
- * Longest substring without repeating characters

Array Programs

- * Largest element
- * Smallest element
- * Second largest
- * Second smallest
- * Reverse array
- * Sort array
- * Remove duplicates
- * Find duplicates
- * Missing number
- * Common elements
- * Merge arrays
- * Rotate array
- * Move zeros
- * Two Sum
- * Maximum subarray
- * Frequency of elements

Collections Coding

- * Find duplicates using Set
- * Count frequency using Map
- * Sort List
- * Sort Map
- * Find max/min
- * Find second highest
- * Remove duplicates
- * Group objects
- * Convert List to Map
- * Iterate Map
- * Comparable vs Comparator coding

Java 8/17/21 Coding

- * Stream-based filtering

- * Mapping
- * Sorting
- * Grouping
- * Partitioning
- * Counting
- * Finding highest/lowest
- * Second highest
- * Department-wise grouping
- * Employee salary problems
- * Optional-based coding

OOP Coding

- * Encapsulation example
- * Inheritance example
- * Method overloading
- * Method overriding
- * Interface implementation
- * Abstract class
- * Runtime polymorphism
- * Composition

Multithreading Coding

- * Create thread
- * Runnable
- * Callable
- * ExecutorService
- * Thread-safe counter
- * Synchronization
- * Producer-consumer
- * CompletableFuture

Coding Explanation Format

For every programming question:

1. Explain the approach.
2. Provide clean Java 17/21 code.
3. Explain important lines.

4. Give time complexity.
5. Give space complexity.
6. Mention important edge cases.
7. If applicable, provide a Java Stream solution also.

3. Spring Framework – Basic to Advanced

Spring Basics

- * Spring Framework
- * Features of Spring
- * IoC
- * Dependency Injection
- * Bean
- * ApplicationContext
- * BeanFactory
- * Dependency injection types
- * Constructor injection
- * Setter injection
- * Field injection

Spring Bean

- * Bean lifecycle
- * Bean scopes
- * Singleton
- * Prototype
- * Request
- * Session
- * @Component
- * @Service
- * @Repository
- * @Controller
- * @RestController
- * @Configuration
- * @Bean

Spring Boot

- * Spring Boot
- * Spring vs Spring Boot
- * Auto Configuration
- * Starter dependencies
- * Embedded server
- * application.properties
- * application.yml
- * Profiles
- * Configuration
- * Actuator
- * Logging
- * Spring Boot lifecycle

Important Annotations

- * @SpringBootApplication
- * @RestController
- * @Controller
- * @Service
- * @Repository
- * @Component
- * @Configuration
- * @Bean
- * @Autowired
- * @Value
- * @Qualifier
- * @Primary
- * @Profile
- * @Transactional

4. REST API – Basic to Advanced

- * REST
- * REST principles
- * RESTful API
- * HTTP

- * HTTP methods
- * GET
- * POST
- * PUT
- * PATCH
- * DELETE
- * HTTP status codes
- * Headers
- * Request body
- * Response body
- * Query parameters
- * Path variables
- * Content-Type
- * Accept
- * JSON
- * Serialization
- * Deserialization
- * Idempotency
- * Statelessness
- * Pagination
- * Sorting
- * Filtering
- * Searching
- * API versioning
- * API documentation

Spring MVC Annotations

- * @RequestMapping
- * @GetMapping
- * @PostMapping
- * @PutMapping
- * @PatchMapping
- * @DeleteMapping
- * @RequestBody

- * @PathVariable
- * @RequestParam
- * @RequestHeader
- * @ResponseBody
- * ResponseEntity

5. DTO and Validation

- * DTO
- * Entity vs DTO
- * Request DTO
- * Response DTO
- * DTO mapping
- * Validation
- * @Valid
- * @Validated
- * @NotNull
- * @NotBlank
- * @NotEmpty
- * @Size
- * @Email
- * @Min
- * @Max
- * @Pattern
- * Custom validation

6. Exception Handling in Spring Boot

- * @ExceptionHandler
- * @ControllerAdvice
- * @RestControllerAdvice
- * Custom exceptions
- * Global exception handling
- * Error response structure
- * HTTP error responses

- * Validation error handling
- * Business exceptions
- * Logging exceptions

7. Spring Security – Basic to Advanced

Security Basics

- * Authentication
- * Authorization
- * Authentication vs Authorization
- * SecurityFilterChain
- * Security Context
- * UserDetails
- * UserDetailsService
- * PasswordEncoder
- * BCrypt
- * Roles
- * Authorities

* RBAC

JWT

- * JWT
- * JWT structure
- * Header
- * Payload
- * Signature
- * JWT generation
- * JWT validation
- * JWT expiration
- * Refresh token
- * Access token
- * JwtFilter
- * AuthenticationManager
- * SecurityContext

Security Concepts

- * Stateless authentication
- * Session management
- * CORS
- * CSRF
- * Password hashing
- * Role-based authorization
- * Method-level security
- * @PreAuthorize
- * @Secured

8. Hibernate / JPA – Basic to Advanced

- * JPA
- * Hibernate
- * JPA vs Hibernate
- * Entity
- * EntityManager
- * Persistence Context
- * Repository
- * JpaRepository
- * CrudRepository
- * Query methods
- * JPQL
- * Native SQL
- * Named queries

Entity Relationships

- * One-to-One
- * One-to-Many
- * Many-to-One
- * Many-to-Many
- * @JoinColumn
- * mappedBy

Fetching

- * Lazy loading

- * Eager loading
- * FetchType.LAZY
- * FetchType.EAGER

Cascade

- * CascadeType.ALL
- * PERSIST
- * MERGE
- * REMOVE
- * REFRESH
- * DETACH

Advanced Hibernate

- * N+1 problem
- * Fetch Join
- * Entity lifecycle
- * First-level cache
- * Second-level cache
- * Dirty checking
- * Persistence context
- * Optimistic locking
- * Pessimistic locking

9. Transactions

- * Transaction
- * @Transactional
- * ACID
- * Commit
- * Rollback
- * Isolation levels
- * Propagation
- * Read-only transaction
- * Transaction boundaries
- * Optimistic locking
- * Pessimistic locking

- * Concurrent update handling

10. MySQL / SQL – Basic to Advanced

SQL Basics

- * Database
- * Table
- * Row
- * Column
- * Primary key
- * Foreign key
- * Unique key
- * NOT NULL
- * DEFAULT
- * CHECK
- * Constraints

Queries

- * SELECT
- * INSERT
- * UPDATE
- * DELETE
- * WHERE
- * ORDER BY
- * GROUP BY
- * HAVING
- * DISTINCT
- * LIMIT

Joins

- * INNER JOIN
- * LEFT JOIN
- * RIGHT JOIN
- * CROSS JOIN
- * Self Join

Advanced SQL

- * Subquery
- * Correlated subquery
- * CTE
- * Aggregate functions
- * Window functions
- * ROW_NUMBER
- * RANK
- * DENSE_RANK
- * CASE
- * EXISTS
- * UNION
- * UNION ALL

SQL Coding

- * Second highest salary
- * Third highest salary
- * Nth highest salary
- * Duplicate records
- * Remove duplicates
- * Department-wise highest salary
- * Top N salaries
- * Employee count
- * Employees without department
- * Multiple joins
- * Grouping
- * Aggregation

11. Database Concepts

- * Normalization
- * 1NF
- * 2NF
- * 3NF
- * Denormalization
- * Index

- * Clustered/non-clustered concepts
- * Composite index
- * Query optimization
- * Execution plan
- * Database transactions
- * ACID
- * Connection pool
- * HikariCP
- * Database locking
- * Deadlock

12. Microservices – Basic to Advanced

- * Monolithic architecture
- * Microservices architecture
- * Advantages
- * Disadvantages
- * Service independence
- * Service discovery
- * Eureka
- * API Gateway
- * Inter-service communication
- * REST communication
- * Feign Client
- * Load balancing
- * Centralized configuration
- * Distributed logging
- * Monitoring
- * Distributed transactions
- * Event-driven architecture
- * Fault tolerance

13. API Gateway

- * API Gateway

- * Routing
- * Authentication
- * Authorization
- * Filtering
- * Rate limiting
- * Logging
- * Load balancing
- * Request transformation
- * Central entry point

14. Load Balancing

- * Load balancing
- * Horizontal scaling
- * Vertical scaling
- * Round Robin
- * Weighted Round Robin
- * Least Connections
- * Health checks
- * High availability
- * Fault tolerance

15. Rate Limiting

- * Rate limiting
- * Requests per second
- * Requests per minute
- * IP-based limiting
- * User-based limiting
- * API-based limiting
- * Token bucket
- * Leaky bucket
- * HTTP 429
- * Distributed rate limiting

16. Throttling

- * Throttling
- * Request processing rate
- * Rate limiting vs throttling
- * Traffic control
- * Backpressure
- * API protection

17. Caching / Redis

- * Cache
- * Cache hit
- * Cache miss
- * Cache-aside pattern
- * TTL
- * Cache invalidation
- * Redis
- * Redis data structures
- * Distributed cache
- * Cache consistency

Spring Cache

- * @Cacheable
- * @CachePut
- * @CacheEvict

18. Circuit Breaker

- * Circuit breaker
- * Failure handling
- * Timeout
- * Retry
- * Fallback
- * CLOSED
- * OPEN
- * HALF_OPEN

- * Resilience4j concepts

19. External API Integration

- * REST client
- * RestTemplate
- * WebClient
- * Feign Client
- * Request headers
- * Authentication
- * Timeout
- * Retry
- * Error handling
- * Third-party API failure
- * API response validation

20. Payment Integration

Understand complete payment architecture:

- * Create order
- * Payment request
- * Payment gateway
- * Payment success
- * Payment failure
- * Payment verification
- * Signature verification
- * Transaction ID
- * Idempotency
- * Duplicate payment
- * Payment status
- * Pending state
- * Retry
- * Webhook concept
- * Secure payment processing

21. WhatsApp / Notification Integration

- * Third-party API integration
- * REST API call
- * Authentication
- * Access token
- * Request/response
- * Webhook
- * OTP
- * Notification
- * Failure handling
- * Retry
- * Logging

22. WebSocket / Real-Time Communication

- * WebSocket
- * HTTP vs WebSocket
- * Persistent connection
- * Full duplex communication
- * Real-time chat
- * Online/offline status
- * Typing indicator
- * Notifications
- * Socket.IO concepts
- * Connection handling
- * Room concept
- * Message delivery
- * Reconnection

23. File Upload

- * Multipart request
- * MultipartFile
- * File validation
- * File size validation

- * File type validation
- * Storage
- * File URL
- * Secure file handling
- * Exception handling

24. Git / GitHub

- * Git
- * GitHub
- * Repository
- * Clone
- * Pull
- * Fetch
- * Add
- * Commit
- * Push
- * Branch
- * Merge
- * Rebase
- * Stash
- * Reset
- * Revert
- * Cherry-pick
- * Pull Request
- * Merge conflict
- * Git workflow

25. Maven

- * Maven
- * pom.xml
- * Dependencies
- * Plugins
- * Maven lifecycle

- * Dependency management
- * Scope
- * clean
- * compile
- * test
- * package
- * install

26. Swagger / OpenAPI

- * Swagger
- * OpenAPI
- * API documentation
- * Request documentation
- * Response documentation
- * Authentication in Swagger
- * Bearer token
- * API testing

27. Postman

- * Collections
- * Environment
- * Variables
- * Headers
- * Authorization
- * Bearer token
- * Request body
- * Tests
- * API testing
- * Different environments

28. Docker

Basic

- * Docker

- * Image
- * Container
- * Dockerfile
- * Docker Compose
- * Port mapping
- * Environment variables

Advanced

- * Container networking
- * Volumes
- * Multi-stage build
- * Container logs
- * Dockerized Spring Boot application

29. AWS / Deployment

- * EC2
- * Server
- * Linux
- * SSH
- * JAR deployment
- * Environment variables
- * Port configuration
- * Firewall
- * Nginx
- * Reverse proxy
- * Domain
- * DNS
- * SSL
- * HTTPS
- * Server logs
- * Application restart
- * Production deployment

30. Production Support

Prepare scenarios for:

- * API is slow
- * Database is slow
- * High traffic
- * Server is down
- * Memory issue
- * CPU high
- * API timeout
- * Third-party API failure
- * Payment failure
- * Duplicate payment
- * Database deadlock
- * Concurrent requests
- * JWT failure
- * CORS issue
- * 500 error
- * 404 error
- * 401 error
- * 403 error
- * 429 error
- * Deployment failure

31. Performance Optimization

- * Query optimization
- * Database indexes
- * Pagination
- * Caching
- * Redis
- * Lazy loading
- * N+1 query prevention
- * Connection pooling
- * Async processing
- * Thread pools

- * Load balancing
- * Rate limiting
- * API response optimization
- * Reducing unnecessary DB calls

32. Design Principles

Prepare:

- * SOLID principles
- * Single Responsibility
- * Open/Closed
- * Liskov Substitution
- * Interface Segregation
- * Dependency Inversion
- * DRY
- * KISS
- * Separation of concerns
- * Loose coupling
- * High cohesion

33. Design Patterns

Basic to advanced:

- * Singleton
- * Factory
- * Builder
- * Strategy
- * Observer
- * Adapter
- * Proxy
- * Template Method
- * Dependency Injection
- * Repository pattern
- * DTO pattern
- * MVC pattern

Understand where each pattern can be used in backend applications.

34. System Design Basics

Prepare:

- * Scalability
- * Availability
- * Reliability
- * Fault tolerance
- * Horizontal scaling
- * Vertical scaling
- * Load balancing
- * Caching
- * Database scaling
- * Read/write separation
- * API Gateway
- * Microservices
- * Queue
- * Asynchronous processing
- * Rate limiting
- * Circuit breaker

Be ready for basic designs such as:

- * URL Shortener
- * E-commerce backend
- * Chat system
- * Auction system
- * Payment system
- * Notification system
- * Loan management system
- * Vehicle marketplace

35. AI / Gemini – Basic to Advanced

Prepare:

- * Artificial Intelligence

- * Machine Learning
- * Generative AI
- * LLM
- * Gemini
- * Prompt
- * Context
- * Context window
- * Input tokens
- * Output tokens
- * Model capability
- * Latency
- * Cost
- * Temperature
- * Structured output
- * Function calling
- * Multimodal AI
- * Embeddings
- * Vector database
- * RAG
- * Fine-tuning
- * Hallucination
- * Prompt injection

RAG

Understand:

```text

Document

↓

Chunking

↓

Embedding

↓

Vector Database

↓

User Query

↓

Similarity Search

↓

Relevant Context

↓

LLM/Gemini

↓

Response

...

### RAG vs Fine-tuning

Understand:

- \* When to use RAG
- \* When to use fine-tuning
- \* Knowledge retrieval
- \* Model behavior customization
- \* Hallucination reduction

---

# 36. Interview Scenario Questions

Prepare practical answers for:

- \* Why interface?
- \* Why DTO?
- \* Why JWT?
- \* Why Spring Boot?
- \* Why Hibernate?
- \* Why JPA?
- \* Why MySQL?
- \* Why Redis?
- \* Why microservices?
- \* Why API Gateway?
- \* Why load balancing?
- \* Why rate limiting?
- \* Why caching?

- \* Why circuit breaker?
- \* Why @Transactional?
- \* Why constructor injection?
- \* Why global exception handling?
- \* Why pagination?
- \* How do you secure APIs?
- \* How do you handle high traffic?
- \* How do you optimize slow APIs?
- \* How do you handle third-party failures?
- \* How do you prevent duplicate payment?
- \* How do you handle concurrent requests?
- \* How do you handle production issues?

---

Which APIs have you developed?

What was your role in the project?

How do you implement JWT authentication?

How do you handle exceptions?

How do you optimize a slow API?

How do you handle database performance?

How did you integrate Razorpay?

How did you integrate WhatsApp API?

How do you coordinate with frontend and testing teams?

What production issue did you face and how did you solve it?

What are your responsibilities?

Answer:

I am working as a Backend Java Developer. My main responsibility is to develop and maintain backend applications using Java, Spring Boot, REST APIs, and MySQL.

I work on designing and developing REST APIs, implementing business logic, database integration, authentication and authorization using JWT, third-party integrations such as payment and WhatsApp APIs, and exception handling.

I also test APIs using Postman and Swagger, debug production and development issues, work with Git and GitHub, and coordinate with frontend and testing teams.

I am also involved in deployment-related activities and API integration with the frontend.

Short answer:

My main responsibilities are developing REST APIs, implementing business logic, database integration, JWT security, third-party API integrations, testing, debugging, and deployment support.

2. Which APIs have you developed?

Answer:

In my projects, I have developed several REST APIs related to user management, authentication, loan processing, vehicle management, document management, payments, subscriptions, chat, and notifications.

For example, I have developed APIs for:

User registration and login

JWT authentication

User and profile management

Loan application

Vehicle details

Document upload and status

Payment and subscription

WhatsApp messaging

Chat and notifications

Admin operations

I use HTTP methods such as GET, POST, PUT, PATCH, and DELETE depending on the requirement.

Example:

POST /api/auth/login

POST /api/auth/register

GET /api/users/{id}

POST /api/loans

PUT /api/loans/{id}

POST /api/payment/create-order

POST /api/whatsapp/send

POST /api/documents/upload

3. Explain your current project.

Answer:

Currently, I am working on an application called Adalat, which is a legal platform. The backend is developed using Java and Spring Boot, and we use MySQL as the database. The frontend is developed using React.

My responsibility is mainly on the backend side. I develop REST APIs, implement business logic, manage database operations, implement authentication and authorization, and integrate frontend requirements with backend APIs.

The project includes modules such as user management, admin management, legal case intake, document handling, chatbot/legal assistant functionality, lawyer matching, and communication-related features.

I have also worked on the deterministic legal intake engine, where the system identifies the legal category based on predefined rules and questions rather than depending completely on an external AI API.

I test the APIs using Postman and Swagger and coordinate with the frontend and testing teams.

If interviewer asks "What exactly did YOU work on?"

My major contribution has been backend API development, business logic implementation, database integration, chatbot/legal intake functionality, DTO design, exception handling, and API testing.

4. Where did you use interfaces?

This is a very important interview question.

Answer:

I use interfaces mainly to achieve abstraction and loose coupling.

For example, in Spring Boot, I create a service interface such as UserService and then provide its implementation using UserServiceImpl.

The controller depends on the interface instead of directly depending on the implementation.

This makes the code easier to maintain, test, and extend.

Example:

```
public interface UserService {
 UserDto getUserById(Long id);
 UserDto createUser(UserDto userDto);
}
```

Implementation:

```

@Service
public class UserServiceImpl implements UserService {
 @Override
 public UserDto getUserById(Long id) {
 // business logic
 }
 @Override
 public UserDto createUser(UserDto userDto) {
 // business logic
 }
}

```

Controller:

```

@RestController
@RequestMapping("/api/users")
public class UserController {
 private final UserService userService;
 public UserController(UserService userService) {
 this.userService = userService;
 }
}

```

Interviewer may ask: Why interface?

It provides abstraction and loose coupling. It also makes unit testing easier because we can mock the interface, and we can provide different implementations if required.

5. Explain JWT implementation.

Answer:

I have implemented JWT-based authentication using Spring Security.

The flow is:

User sends username/email and password to the login API.

Spring Security validates the credentials.

If the credentials are valid, I generate a JWT token.

The token contains information such as username and roles and has an expiration time.

The client sends this token in the Authorization header for subsequent requests.

A JWT filter intercepts the request and extracts the token.

The token is validated.

If the token is valid, the authenticated user is placed into the Spring Security context.

Based on the user's role and authorization rules, the request is allowed or rejected.

Header:

Authorization: Bearer

Security configuration:

http

```
.csrf(csrf -> csrf.disable())
```

```
.sessionManagement(session ->
```

```
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
```

```
)
```

```
.authorizeHttpRequests(auth -> auth
```

```
.requestMatchers("/api/auth/**").permitAll()
```

```
.anyRequest().authenticated()
```

```
);
```

JWT Filter:

I use a filter that extends `OncePerRequestFilter`. It reads the Authorization header, extracts the token, validates it, extracts the username, and sets authentication in `SecurityContextHolder`.

6. Explain payment integration.

Answer:

I have worked with payment integration using Razorpay.

The basic flow is:

User selects a product, premium service, or subscription.

Backend calculates the amount.

Backend creates a Razorpay order.

Razorpay returns an order ID.

The order ID is sent to the frontend.

The frontend opens the Razorpay payment interface.

After successful payment, Razorpay provides payment details.

Backend verifies the payment signature.

After successful verification, we update the payment status in our database.

Then we activate the related service, subscription, or premium feature.

Example:

User



Backend



Create Razorpay Order



Frontend Payment Page



Razorpay



Payment Success



Backend Verification



Update DB



Activate Service

Important interview point:

I don't directly trust the payment success response from the frontend. I verify the payment signature on the backend before updating the payment status.

7. Explain WhatsApp API integration.

Answer:

I have worked with the WhatsApp Cloud API provided by Meta.

The backend receives the phone number and message from the application. Then the backend calls the Meta WhatsApp API using an HTTP client such as RestTemplate.

We configure the WhatsApp access token, phone number ID, and Graph API endpoint in the application properties.

The backend sends the message to Meta, and Meta delivers it to the user's WhatsApp number.

I also worked with callback/webhook configuration for receiving events from WhatsApp.

Flow:

Application

↓

Spring Boot API

↓

WhatsApp Service

↓

Meta WhatsApp Cloud API

↓

User's WhatsApp

Example endpoint:

POST /api/v1/whatsapp/send

Request:

```
{
 "phoneNumber": "XXXXXXXXXX",
 "message": "Your application has been approved."
}
```

Interview point:

I keep the access token and configuration values in application properties or environment variables instead of hardcoding them in the source code.

8. How do you handle exceptions?

Answer:

I handle exceptions using centralized exception handling with `@RestControllerAdvice` and `@ExceptionHandler`.

Instead of writing try-catch blocks in every controller, I create custom exceptions for business-related errors and handle them globally.

For example, if a user is not found, I throw `UserNotFoundException`. The global exception handler catches it and returns an appropriate HTTP status and error response.

Example:

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class UserNotFoundException extends RuntimeException {
 public UserNotFoundException(String message) {
 super(message);
 }
}
```

Global handler:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
 @ExceptionHandler(UserNotFoundException.class)
 public ResponseEntity handleUserNotFound(
 UserNotFoundException ex) {
 return ResponseEntity
 .status(HttpStatus.NOT_FOUND)
 .body(ex.getMessage());
 }
}
```

Response:

```
{
 "status": 404,
 "message": "User not found"
}
```

Short answer:

I use custom exceptions and centralized exception handling using `@RestControllerAdvice`, which gives consistent error responses across the application.

9. How do you test APIs?

Answer:

I mainly use Postman and Swagger for API testing.

First, I test the positive scenario with valid input. Then I test negative scenarios such as invalid input, missing fields, invalid JWT, unauthorized access, duplicate data, and resource-not-found cases.

I verify the HTTP status code, response body, headers, and database changes.

For example:

- 200 OK – successful GET/update operation
- 201 Created – successful resource creation
- 400 Bad Request – invalid input
- 401 Unauthorized – missing or invalid authentication
- 403 Forbidden – insufficient permission
- 404 Not Found – resource doesn't exist

500 Internal Server Error – unexpected server error

Example:

POST /api/auth/login

↓

Send email + password

↓

Check response

↓

Receive JWT

↓

Use JWT for protected APIs

↓

Verify response + DB

10. What challenges did you face?

This is another very important question.

Answer:

I have faced several challenges during backend development.

One challenge was integrating third-party services such as payment and WhatsApp APIs because we had to handle authentication, API responses, failures, and callback verification properly.

Another challenge was implementing JWT security and role-based authorization while making sure that protected APIs were accessible only to authorized users.

I also faced database-related issues such as incorrect queries, relationship mapping, and transaction-related problems.

During development, I also handled issues related to CORS, API integration with the frontend, exception handling, and deployment configuration.

My approach is to reproduce the issue, check application logs and stack traces, identify the root cause, fix it, and then test the complete flow using Postman and the application.

★ Very Important Follow-up Questions

The interviewer may ask these immediately after your answers.

11. What is the difference between Controller and RestController?

@Controller is generally used for MVC applications where the method can return a view.

@RestController is used for REST APIs. It combines @Controller and @ResponseBody, so the returned object is directly converted into JSON or XML.

12. What is @RequestBody?

@RequestBody is used to map the HTTP request body JSON into a Java object.

Example:

```
@PostMapping("/users")
public User createUser(@RequestBody User user) {
 return userService.createUser(user);
}
```

13. What is Dependency Injection?

Dependency Injection means providing the required dependency to a class instead of creating the dependency manually using new.

In Spring Boot, the Spring container manages the objects and injects them where required.

I generally prefer constructor injection.

14. What is Spring Boot?

Spring Boot is a framework built on top of Spring that simplifies application development by providing auto-configuration, starter dependencies, embedded servers, production-ready features, and less configuration.

★ 15. What is REST API?

REST API is an architectural style used for communication between client and server over HTTP.

It uses HTTP methods such as GET, POST, PUT, PATCH, and DELETE and generally exchanges data in JSON format.

★ 16. Difference between PUT and PATCH?

PUT is generally used to replace or update the complete resource, while PATCH is used for partial updates.

Example:

PUT /users/10

can update the complete user.

PATCH /users/10

can update only the user's phone number.

★ 17. What is DTO and why do you use it?

DTO stands for Data Transfer Object.

I use DTOs to transfer only the required data between the client and backend instead of directly exposing entity objects.

It improves security, maintains separation between API and database models, and gives better control over request and response structures.

#### ★ 18. What is JPA/Hibernate?

JPA is a Java specification for ORM, while Hibernate is one of the popular implementations of JPA.

It allows us to map Java objects to database tables and perform database operations using entities and repositories instead of writing SQL for every operation.

#### ★ 19. What is @Transactional?

@Transactional is used to manage database transactions.

If multiple database operations are part of one business operation, they can execute within the same transaction. If an appropriate failure occurs, the transaction can be rolled back.

Example:

@Transactional

```
public void processPayment() {
 // update payment
 // activate subscription
}
```

#### ★ 20. Why do you use ResponseEntity?

ResponseEntity gives me control over the HTTP response, including status code, headers, and response body.

Example:

```
return ResponseEntity
 .status(HttpStatus.CREATED)
 .body(user);
```

#### ★ 21. How do you secure APIs?

I secure APIs using Spring Security and JWT.

Public endpoints such as registration and login are permitted without authentication.

Protected endpoints require a valid JWT token.

I also implement role-based authorization where required.

★ 22. What happens if JWT is expired?

The JWT validation fails because the token's expiration time has passed. The request is rejected and the API returns an unauthorized response, generally HTTP 401.

★ 23. What is Microservices architecture?

Microservices architecture divides an application into small, independent services. Each service focuses on a particular business capability and can be developed, deployed, and scaled independently.

Services communicate using APIs or messaging mechanisms.

★ 24. What is Eureka?

Eureka is a service discovery mechanism commonly used in Spring Cloud applications. Microservices register themselves with the Eureka Server, and other services can discover them using the service name instead of hardcoding IP addresses and ports.

★ 25. What is API Gateway?

API Gateway acts as a single entry point for clients.

It can handle routing, authentication, authorization, filtering, logging, and other cross-cutting concerns before forwarding requests to backend services.

Yes, this is a good real project challenge to explain in the interview. You can present it clearly like this:

Chatbot Challenge Answer

Another challenge I faced was in the chatbot functionality of our project.

Initially, when the user asked a question, the chatbot was not always giving the correct or relevant answer. Sometimes the answer was incomplete or the chatbot misunderstood the user's response.

For example, during the legal-question flow, we were asking users questions like Yes/No or selecting options. The chatbot needed to understand responses such as "Yes", "No", "Okay", or similar variations. Initially, these responses were not being handled properly.

Another issue was with the conversation flow. After the user answered around 3–4 questions, the chatbot sometimes lost the conversation state and started asking the first question again. This created a loop and resulted in a poor user experience.

I analyzed the conversation flow and found that we needed better response validation and proper conversation-state management. I improved the handling of Yes/No and similar responses, added proper validation and maintained the current question/state so that the chatbot could continue from where the user left off instead of restarting.

After making these changes, I tested different conversation scenarios, including valid answers, invalid answers, Yes/No responses, and multiple questions in sequence, to make sure the chatbot maintained the correct flow.

If interviewer asks: "How did you solve it technically?"

You can say:

I maintained the conversation state on the backend. For each conversation, I stored the current question or stage and the user's previous responses. Before generating the next question, the system checked the current conversation state instead of starting from the first question. I also added response validation for Yes/No and similar user inputs. This prevented the conversation from going into a loop.

Very short version for interview

I faced an issue where the chatbot was giving irrelevant answers and, after 3–4 questions, it sometimes restarted from the first question. I solved it by improving response validation and maintaining the conversation state on the backend. I stored the current question and previous responses, so the chatbot could continue from the correct step instead of restarting.

mag mi thieh ai cya services use kelua

हो. जर तू chatbot साठी external AI service/API वापरली असेल, तर interview मध्ये ते clearly सांग. पण तू कोणती service वापरली हे exact सांगितल्याशिवाय मी नाव assume करणार नाही.

तुझ्या case मध्ये answer असा frame करू शकतेस:

AI Service Chatbot Answer

In the chatbot module, I used an AI service to process the user's questions and generate relevant responses.

Initially, we faced an issue where the chatbot was not always giving the expected answer. For example, when the user replied with simple inputs like "yes", "no", "okay", or similar responses, the system sometimes failed to understand the context correctly.

We also faced a conversation-state issue. After asking 3–4 questions, the chatbot sometimes lost the previous conversation context and started asking the first question again.

To solve this, I worked on the conversation flow and context management. I made sure that the current question, previous responses, and conversation state were maintained properly and passed to the AI service when required.

I also added proper validation for responses such as yes/no and handled unexpected inputs. After the changes, I tested multiple conversation flows to verify that the chatbot continued from the correct question instead of restarting.

जर interviewer ने विचारलं: “Which AI service did you use?”

तेव्हा exact service चे नाव सांग — उदा. Gemini API, OpenAI API, Claude API इ. तू मला service चे नाव सांग, मी त्यावर “How did you integrate it?” + technical flow + code-level interview answer तयार करून देतो.

Gemini API,

हो, मग Gemini API exact सांग. Interview मध्ये असा answer दे:

### Gemini API Chatbot Challenge

Another challenge I faced was in the chatbot module, where we integrated the Gemini API.

Initially, the chatbot was not always providing the expected or relevant answer. For example, when the user replied with simple responses like "Yes", "No", "Okay", or similar inputs, Gemini sometimes did not understand the response correctly in the context of the previous question.

We also faced a conversation-flow issue. After asking 3–4 questions, the chatbot sometimes lost the previous conversation context and started asking the first question again.

To solve this, I worked on conversation context and state management. We maintained the current question, previous user responses, and conversation stage on the backend.

We then provided the relevant context along with the user's latest question to the Gemini API.

I also added validation for responses such as Yes/No and handled unexpected inputs before sending them to Gemini.

After implementing these changes, I tested multiple conversation flows with different user responses to make sure the chatbot maintained the correct context and continued from the current question instead of restarting.

If interviewer asks: “How did you integrate Gemini API?”

Say:

We integrated Gemini API from the Spring Boot backend. The backend receives the user's message, prepares the required prompt along with the relevant conversation context, and sends the request to the Gemini API. We receive the generated response

from Gemini, process it according to our business logic, and return the final response to the frontend.

Simple flow याद ठेव:

User



Frontend Chat



Spring Boot Chat API



Conversation State + Previous Context



Gemini API



AI Response



Backend Processing



Frontend



User

If they ask: "Why did you use Gemini API?"

We used Gemini API because it provides natural-language understanding and response generation, which helped us handle user questions and conversational interactions more naturally. We combined Gemini's response generation with our own backend business rules and conversation-state management.

Important ⚠

जर interviewer म्हणाला "Did Gemini itself manage the conversation?", don't say yes.

Say:

No. We managed the conversation state on our backend. Gemini was used for understanding and generating responses, while our application controlled the conversation flow, questions, validation, and business logic.

★ 26. How do you debug an issue?

Answer:

First, I reproduce the issue.

Then I check the API request and response, application logs, stack trace, database records, and relevant code.

I identify the root cause, fix the issue, and test the affected functionality again.

After that, I also perform regression testing to make sure the fix hasn't affected existing functionality.

★ 27. How do you work with the frontend team?

I discuss the API requirements with the frontend team and define the request and response structure.

I provide API documentation through Swagger and share endpoint details, request parameters, authentication requirements, and response formats.

I also test the APIs with Postman before integrating them with the frontend.

★ 28. Why should we hire you?

Interview-ready answer:

I have practical experience in Java, Spring Boot, REST APIs, MySQL, Spring Security, JWT, and third-party integrations.

I have worked on real project requirements rather than only theoretical concepts. I am comfortable developing APIs, debugging issues, working with databases, testing APIs, and coordinating with frontend and testing teams.

I am also continuously improving my technical skills and I am comfortable learning new technologies based on project requirements.

🔥 60-Second Introduction

If Infosys starts with "Tell me about yourself", you can say:

Interview Introduction

Good morning. My name is Geetanjali Khaladkar. I am a Backend Java Developer with 2+ years of experience.

My primary technical skills are Core Java, Advanced Java, Spring, Spring Boot, REST APIs, Spring Security, JWT, Hibernate/JPA, MySQL, and Git.

In my current role, I work mainly on backend development. My responsibilities include developing REST APIs, implementing business logic, database integration, authentication and authorization, exception handling, API testing, and third-party integrations such as payment and WhatsApp APIs.

I have worked on projects involving finance, vehicle, bidding, communication, and legal-domain applications. Currently, I am working on the Adalat project, where I am involved in backend API development and legal application functionality.

I use tools such as Postman and Swagger for API testing and documentation, and I work closely with frontend and testing teams.

I am interested in building scalable backend applications and improving my skills in Java and Spring Boot.

For the Inf

### # 37. Interview Answer Rules

For every question:

- \* Start with a direct answer.
- \* Explain the concept in simple English.
- \* Give a practical example.
- \* If relevant, connect it with my resume/project.
- \* For difference questions, clearly compare both concepts.
- \* For "Why" questions, explain the practical reason.
- \* For "How" questions, explain implementation flow.
- \* For coding questions, provide Java 17/21 code.
- \* Explain time and space complexity for coding problems.
- \* Mention edge cases when relevant.
- \* Do not invent experience.
- \* Do not invent project details.
- \* Do not claim technologies that are not supported by my resume.
- \* Keep answers appropriate for approximately 2.5+ years of experience.
- \* Prefer practical backend examples over purely theoretical explanations.
- \* Keep spoken answers concise enough for an actual interview.
- \* If the interviewer asks a follow-up, answer only the follow-up.
- \* Use professional but simple English that I can speak naturally.

The objective is to prepare me for a \*\*Java Backend Developer / Spring Boot Developer interview from Basic to Advanced level\*\*, including theory, programming, SQL, Spring Boot, Security, Hibernate/JPA, Microservices, API design, performance, deployment, production scenarios, and AI/Gemini concepts.